



JAVA PROGRAMMING (340)

STATE – 2021

PRODUCTION PORTION:

Stock Portfolio Text Import _____ (355 points)

TOTAL POINTS _____ (*355 points*)

Test Time: 90 minutes

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder.
3. Submit your entire solution/project so that the graders may open your project to review the source code.
4. Ensure that the files required to run your program are present and will execute on the flash drive provided.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

*It is recommended that you use relative paths rather than absolute paths to ensure that the program will run regardless of the flash drive letter.

The graders will *not* compile or alter your source code to correct for this.
Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- The input file will contain only ASCII characters.
- A test input file will be available and named, "stocksText.txt."
- The Stocks.class file needs to be in your project folder and will be used to help with the solution. This class will be used to construct Stock objects in the Portfolio.

Development Standards:

- Your Code must use a consistent variable naming convention.
- All methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any).
- If you create a class, then you must use Javadoc comments.

Note to Graders: The contestants have been given a file named stocksText.txt. You must first delete that file from the contestant's flash drive and run the program for Test case #1. This should result in a message that the file was *not* found. Then replace the file on their flash drive with the grader's file with the same name and run Test case #2. You'll run their program with a grader's file named stocksText.txt which will contain different data than the student's file. This will produce different results than the student's file. You must copy the grader's stocksText.txt file into the folder where their program runs. Listed below are the test cases and their results.

Test case #1 verifies that the program checks for the correct input file. The output wording does *not* need to be exact.

Text case #2 processes the correct data file name and produces output. The output wording *must* be exact.

Test Case #1

The text file was not found.

Test Case #2

This is your current portfolio:

Symbol: DEF | Company Name: Comp1 | Price: 100.0 | Total Shares: 5 | Total Value: 500.0

Symbol: GHI | Company Name: Comp2 | Price: 10.1 | Total Shares: 10 | Total Value: 101.0

Symbol: JKL | Company Name: Comp3 | Price: 5.0 | Total Shares: 5 | Total Value: 25.0

Symbol: MNO | Company Name: Comp4 | Price: 2.22 | Total Shares: 2 | Total Value: 4.44

This is the total value of the portfolio: \$630.44

MEMBER ID:

Your application will be graded on the following criteria:

Solution and Project

The project is present on the flash drive	_____	10 points
The projects main class is named StockDriver	_____	10 points
The projects helper class is named Portfolio	_____	10 points
Javadoc comments are used for new classes	_____	10 points

Program Execution

The program runs from the USB flash drive	_____	15 points
---	-------	-----------

If the program does *not* execute, then the remaining items in this section receive a score of zero.

The StockDriver class runs and reads the input file	_____	30 points
The StockDriver class displays an error message if the file cannot be found	_____	20 points
The StockDriver class stores data fields from stocksText into proper variables	_____	20 points
The StockDriver class stops reading the input file when “STOP” is found	_____	20 points
Program displays the correct output in the correct order for the records and the field with the formatting from the Stocks class	_____	30 points
Program displays the correct total value of the portfolio	_____	20 points

Source Code Review

The source code is properly commented		
A comment containing the contestant number is present	_____	10 points
Methods and code sections are commented	_____	20 points
The StockDriver class creates Portfolio object stores data from stocksText into ArrayList of Stocks with boughtStocks () method	_____	30 points
Code uses try... catch for exception handling StockDriver class	_____	20 points
The Portfolio class constructor creates an ArrayList of Stocks	_____	30 points
The Portfolio class method getPortfolioTotalValue() calculates and returns the total value of all stocks (it will need to traverse the Stocks ArrayList	_____	20 points
The Portfolio class method toString() returns the total printout for all stocks (it will need to traverse the Stocks ArrayList and access each stocks printStock() method)	_____	20 points
Code uses a consistent variable naming convention	_____	10 points

Total Points: 355

Suggested Solution

```
1 /**
2  * BPA Java Programming Contest : Stock Portfolio Text Import
3  *
4  * @author - contestant number goes here
5  *
6  * @version January 2020
7  *
8  */
9
10
11 import java.io.FileNotFoundException;
12 import java.io.File;
13 import java.util.Scanner;
14
15 public class StockDriver{
16
17     public static void main(String [] args) throws FileNotFoundException
18     {
19
20         //Variable declaration and creation of Portfolio object
21         Portfolio port1 = new Portfolio();
22         String stockSymbol;
23         String stockName;
24         double stockPrice;
25         int stockVolume;
26
27         try {
28             //Access the file via Scanner
29             File text = new File("stocksText.txt"); //FILE IO
30             Scanner sc = new Scanner(text);
31
32             //Accesses first item stores in variable to verify STOP flag is not at top of the
33             list
34             stockSymbol = sc.nextLine();
35
36             //Loops through the entire file until flag STOP is reached
37             while(!stockSymbol.toUpperCase().equals("STOP")) {
38                 stockName = sc.nextLine();
39                 stockPrice = sc.nextDouble(); //assigns to a double variable
40                 sc.nextLine();
41                 stockVolume = sc.nextInt(); //assigns to a int variable
42                 sc.nextLine();
43                 //Passes values to portfolio to create the stock
44                 port1.boughtStock(stockSymbol, stockName, stockPrice, stockVolume);
45                 stockSymbol = sc.nextLine();
46             }
47
48             System.out.print(port1); //This will print the portfolio
49             sc.close();
50         } catch (FileNotFoundException e) {System.out.println("The text file was not found.");}
51     }
52
53 }
```

```
1 import java.util.Scanner;
2
3
4 public class Portfolio
5 {
6
7     //Initialize the variables
8     private ArrayList<Stocks> stocks; //DATA STRUCTURES
9     private int size;
10
11     public Portfolio()
12     {
13         stocks = new ArrayList<Stocks>();
14         size = 0;
15     }
16
17
18
19     //Creates the stock object to be stores in the list. Info passed from StockDriver
20     public void boughtStock(String sym, String nam, double pri, int sha)
21     {
22         Stocks temp = new Stocks(sym, nam, pri, sha);
23         stocks.add(temp);
24         size++;
25     }
26
27
28     //Adds up the total value of the stocks (the calculation for the value of the stock
ownership
29     //is performed in the stock object, getValue() returns this information)
30     private double getPortfolioTotalValue()
31     {
32         double tempValue = 0;
33         for (Stocks s : stocks)
34         {
35             tempValue += s.getValue();
36         }
37         return tempValue;
38     }
39
40     //
41
42     //Overwrite the toString to print the entire portfolio of stocks
43     //printStock() is a object method from Stocks that prints the format for the output
44     public String toString()
45     {
46         String portfolioPrinter = "";
47         for (int a = 0; a < size; a++)
48             portfolioPrinter = portfolioPrinter + stocks.get(a).printStock() + "\n";
49         return "This is your current portfolio:\n" + portfolioPrinter + "\nThis is the total
value of the portfolio: $" + getPortfolioTotalValue();
50     }
51
52
53 }
```